AML-Assignment#6
(Reinforcement Learning (PyRLSimbot))

- Your task is …

> ***Using reinforcement learning technique, please write a program with a learning capacity; so that the robot has to learn by itself exploring around the obstacles and eating the food if it can***.

Please do it wisely and the most importance thing is please do it by yourself. Let's have fun with it. Don't worry about the score toward your robot performance. I love to see you efforts in solving the problem the most. At the end, you will learn a lot, believe me. ☺

In this assignment, you will have to apply Q learning approach to let the simulation robot learn to find food and survive in its environment by itself. The task and the robot are the same as the previous simulation robot assignment. First, you have to define the state of the robot so that Q learning can be applied. You also have to design how to provide the robot with appropriated reward in every action. During the process, sometimes the robot tries actions by random and updates its Q values by its received rewards, and sometimes the robot uses its Q values to choose its actions by itself. After a certain time of running, the updated Q values are then can be used to select actions to perform the task appropriately. In another words, ***it will be seemed like the robot learn to do its tasks by itself from nothing***. ☺

The appropriate way in applying Q learning is listed as follows…

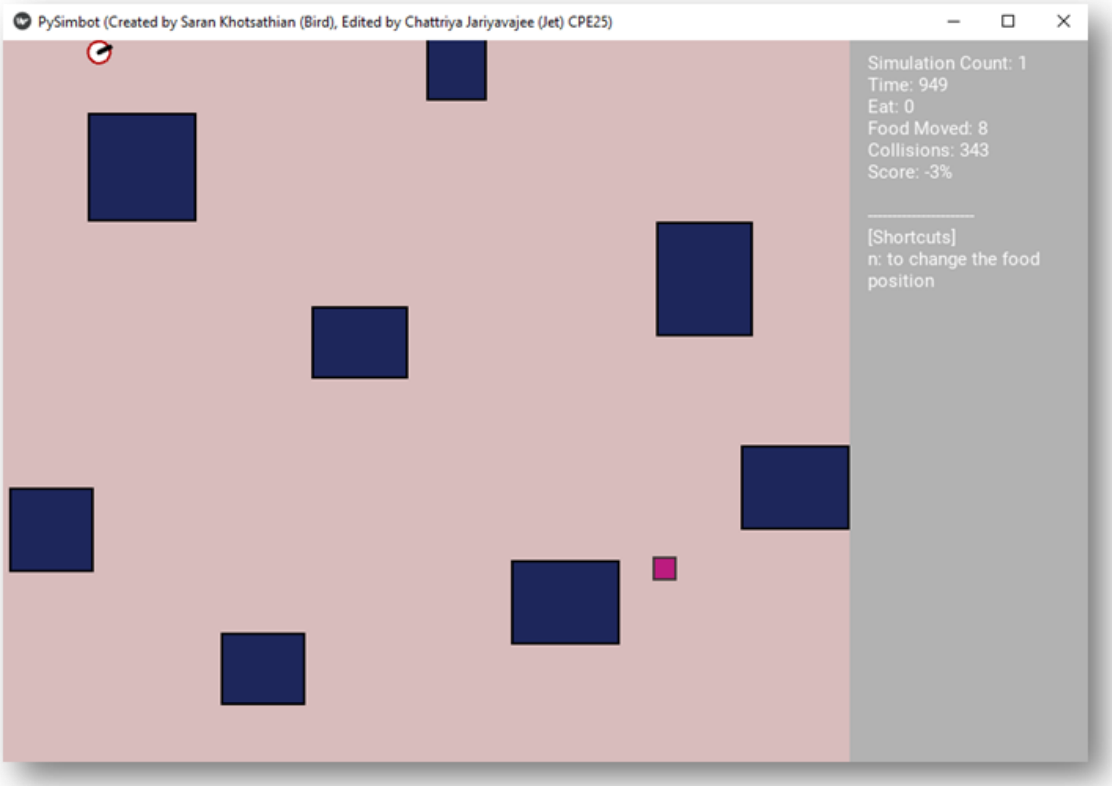
1) Define all possible states of the robot
2) Define all possible actions of the robot
3) Then all Q values can be created as an array of all states and all actions
4) Set all initial values to be zero
5) Set appropriated rewards for each action in every conditions
6) Start the program as follow …

- **Initialize**
  - $Q(s,a)$ for all s, a to 0,
  - $\alpha = 0.5$,
  - $\gamma = 0.9$
- **Do forever:**
  - Observe current state s_t
  - 90 % of time choose an action that has maximum $Q(s_t, a_t)$, 10 % choose some other action. Action is a_t.
  - Carry out action a_t in the world. New state is s_{t+1}
  - Immediate reward for carrying out a in s_t in r_{t+1}
  - Update Q: $\Delta Q = \alpha[rt+1 + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$ for state s_t

7) Q values are updated as follow…

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

8) Run until the task is achieved
9) End



Exploration policy

- **Wacky approach (exploration)**: act randomly in hopes of eventually exploring entire environment
- **Greedy approach (exploitation)**: act to maximize utility using current estimate
- **Reasonable balance**: act more wacky (exploratory) when agent has little idea of environment; more greedy when the model is close to correct

How to define state of the robot?

A state holds all of the information required to predict an action and to determine the reward. The robot has its own sensors as its state of its world. Therefore, you can define the robot's states from the robot's sensors. For example, if the robot has 3 sensors. Each sensor has 3 values. Thus, there are 9 all possible states.

Q-learning is a specific kind of reinforcement learning that assigns values to action-state pairs. The state of the organism is a sum of all its sensory input, including its body position, its location in the environment, the neural activity in its head, etc. So in Q-learning, this means that because for every state there are a number of possible actions that could be taken, each action within each state has a value according to how much or little rewards the organism will get for completing that action (and reward means survival). … ***(Mike Gasser, Tom Busey, Teresa Pegors)***